

# **Virtual Keyboard Using Computer Vision**

Camera Based Touchless Text-Input System

## **Authors**

Nadir Askarov

Fazil Behbudov

Mehdi Hasanli

Zeynalabdin Huseynli

## **Supervisor**

Javid Guliyev

## **Institution**

French-Azerbaijani University (UFAZ)

Université de Strasbourg

## **GitHub Repository**

<https://github.com/nadir2609/Virtual-Keyboard>

## Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Summary / Introduction to the Problem</b>            | <b>2</b> |
| <b>2</b> | <b>Introduction / Goals and Background</b>              | <b>2</b> |
| 2.1      | Goals . . . . .                                         | 2        |
| 2.2      | Background: Computer Vision and Hand Tracking . . . . . | 2        |
| <b>3</b> | <b>Methodology and Results</b>                          | <b>4</b> |
| 3.1      | System Architecture . . . . .                           | 4        |
| 3.2      | Hand Landmark Detection . . . . .                       | 5        |
| 3.3      | Virtual Keyboard Overlay . . . . .                      | 6        |
| 3.4      | Click Detection Algorithm . . . . .                     | 6        |
| 3.5      | Key Injection . . . . .                                 | 7        |
| 3.6      | Results and Discussion . . . . .                        | 7        |
| <b>4</b> | <b>Conclusion</b>                                       | <b>8</b> |
|          | <b>References</b>                                       | <b>9</b> |

## 1. Summary / Introduction to the Problem

Physical keyboards are ubiquitous in computing, yet they impose fundamental constraints on usability. They require direct contact, accumulate pathogens on shared surfaces, and cannot adapt to restricted or mobile environments such as operating rooms, clean rooms, or augmented-reality (AR) workspaces. Users with motor disabilities also face significant barriers when interacting with conventional input devices (Patle et al., 2025).

This project addresses these limitations by designing and implementing a **camera-based virtual keyboard** that requires no physical contact. The system uses a standard RGB webcam to capture a live video feed, detects the user's hand in real time using the MediaPipe Hands framework (Zhang et al., 2020), and interprets specific finger positions and gestures as key-press events. Characters are then injected into the operating system through the `pynput` library, making the virtual keyboard a transparent replacement for any physical keyboard.

The core technical challenge is three-fold: (i) reliably detecting and tracking 21 hand landmarks at interactive frame rates under varied lighting, (ii) mapping continuous hand pose data to discrete keyboard actions without accidental activations, and (iii) rendering a responsive visual interface overlay that gives the user clear spatial feedback. The project demonstrates that these goals are achievable on commodity hardware using only open-source libraries, making the system immediately deployable in real environments.

## 2. Introduction / Goals and Background

### 2.1. Goals

The objectives of this project are:

1. To build a fully functional, real-time virtual keyboard operable with a single RGB webcam.
2. To integrate MediaPipe Hands for robust, low-latency hand-landmark detection (21 landmarks per hand).
3. To implement a reliable click-detection algorithm based on fingertip proximity without false positives.
4. To render an on-screen QWERTY keyboard overlay using OpenCV that provides visual feedback to the user.
5. To evaluate system performance in terms of frame rate, detection accuracy, and subjective usability.

### 2.2. Background: Computer Vision and Hand Tracking

Computer vision-based interaction has grown rapidly as an alternative to physical input devices. Kölsch and Turk (2002) conducted an early survey of virtual keyboards and concluded that the keyboard metaphor – mapping discrete sensing areas to individual characters – remains the most familiar and efficient modality for alphanumeric input. Their analysis showed that ten-finger touch-typing achieves bandwidths of up to 150 words per minute for proficient users, a

target that any virtual alternative must approach to be practical.

The key enabling technology for this project is **MediaPipe**, a cross-platform framework for building perception pipelines from modular, GPU-accelerated calculator components (Lugaresi et al., 2019). MediaPipe represents a perception pipeline as a directed graph of *Calculators* connected by typed data *Streams* carrying timestamped *Packets*. This architecture provides deterministic synchronization, scalable GPU support, and a rich set of reusable components for media processing and model inference. Figure 1 shows an example MediaPipe graph for real-time object detection, illustrating how *FrameSelection*, *DetectionTracking*, *ObjectDetection*, *DetectionMerging*, and *DetectionAnnotation* calculators are composed into a single end-to-end pipeline from camera input to display output. The virtual keyboard in this project follows the same graph-based paradigm, replacing object detection with hand-landmark regression.

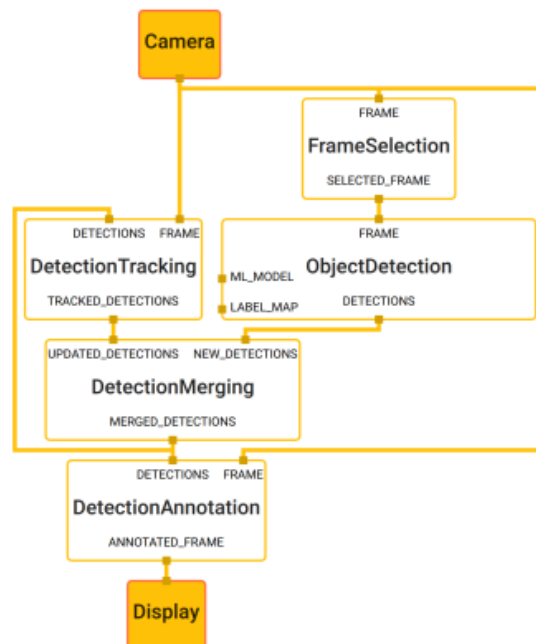


Figure 1: Example MediaPipe perception pipeline for object detection. Transparent boxes are calculator nodes; solid boxes are external inputs/outputs; edges carry typed, timestamped data packets (Lugaresi et al., 2019).

On top of MediaPipe, the **MediaPipe Hands** solution (Zhang et al., 2020) provides a two-stage pipeline:

1. A **BlazePalm detector** that locates palm regions in the full image using a single-shot detector with an encoder-decoder feature pyramid and focal loss training. The detector operates only on the first frame or when tracking is lost, saving significant computation.
2. A **Hand Landmark model** that regresses 21 three-dimensional landmarks from the cropped palm region. The model simultaneously outputs (a) the 21 landmark coordinates with relative depth, (b) a hand-presence confidence score, and (c) a left/right handedness classification.

The 21 landmarks follow a standardised topology, illustrated in Figure 2. Each finger is described by four joints: MCP (metacarpophalangeal), PIP (proximal interphalangeal), DIP (distal

interphalangeal), and TIP, plus the wrist (landmark 0) and the thumb’s CMC and IP joints.

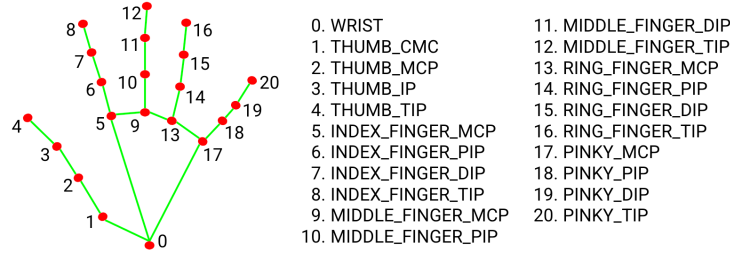


Figure 2: MediaPipe Hands landmark topology. Each red dot represents one of the 21 tracked points (Zhang et al., 2020).

Table 1 reproduces the performance characteristics reported by Zhang et al. (2020) for three model variants on different mobile devices.

Table 1: Hand landmark model performance characteristics (Zhang et al., 2020).

| Model | Params (M) | MSE   | Pixel 3 (ms) | iPhone 11 (ms) |
|-------|------------|-------|--------------|----------------|
| Light | 1.00       | 11.83 | 6.6          | 1.1            |
| Full  | 1.98       | 10.05 | 16.1         | 5.3            |
| Heavy | 4.02       | 9.817 | 36.9         | 7.5            |

The Full model provides the best balance between speed and accuracy and was selected for this project. Training data combined real-world images (6 000 in-the-wild and 10 000 in-house gesture images) with 100 000 synthetic frames, reducing mean squared error from 16.1% (real-world only) to 13.4% when both sets are used together (Zhang et al., 2020).

**OpenCV** (Bradski, 2000) provides the image acquisition loop, color-space conversions, rendering of the keyboard overlay, and frame display. Together, MediaPipe and OpenCV form the complete perception and rendering stack.

**Touchless interfaces** extend beyond keyboards. Patle et al. (2025) review the broader landscape of touchless HCI, covering gesture, voice, and gaze input modalities, and evaluate them across healthcare, education, and industrial settings. Their work confirms that gesture-based interaction is the most reliable modality in quiet, well-lit environments – precisely the conditions a virtual keyboard targets.

### 3. Methodology and Results

#### 3.1. System Architecture

The virtual keyboard system is implemented in Python and organised in following modules

- **Camera module** – Initialises the OpenCV `VideoCapture` object, configures resolution (typically  $1280 \times 720$ ), and manages the main frame loop.
- **Hand tracking module** – Uses the MediaPipe Tasks Hand Landmarker API (`mediapipe.tasks.py`

to detect hand landmarks and exposes helper functions for extracting fingertip and joint coordinates.

- **Keyboard layout module** – Defines the QWERTY layout as a list of rows, each row containing key labels with their bounding-box coordinates in the overlay image.
- **Click detector** – Measures the Euclidean distance between the index-finger tip (landmark 8) and the thumb tip (landmark 4) to distinguish a hover state from a click event.
- **Key injector** – Uses `pynput.keyboard.Controller` to emit key events in the host operating system.

The high-level data flow is shown in Figure 3.

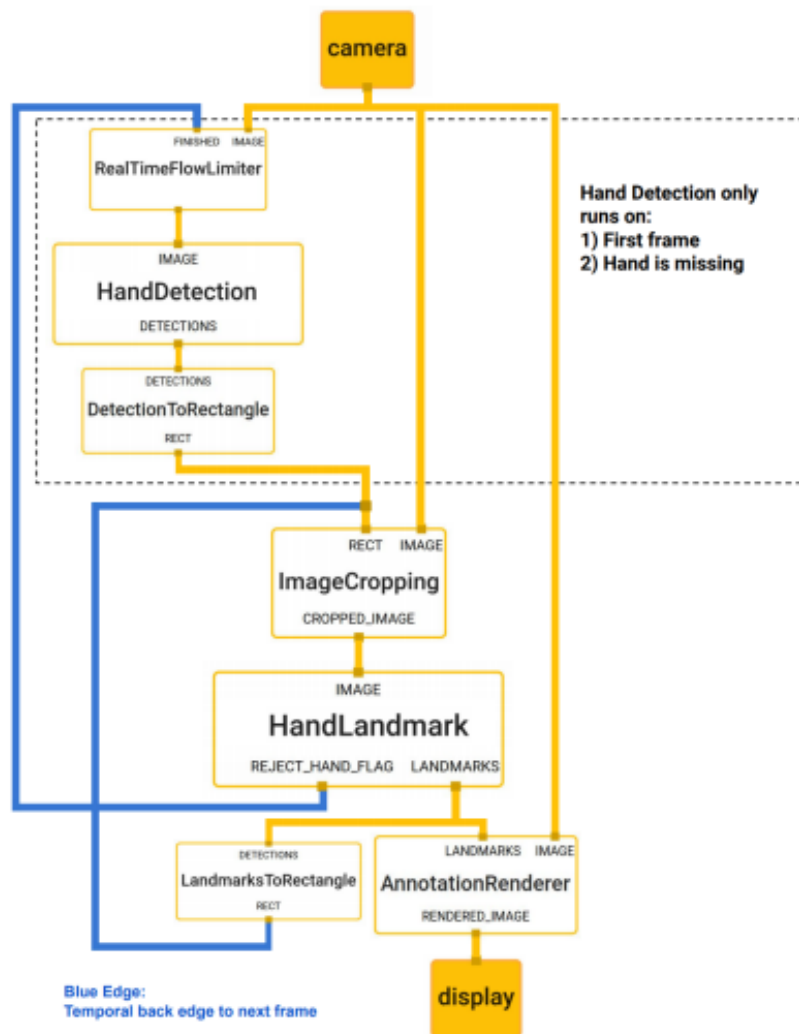


Figure 3: MediaPipe Hands pipeline used by the virtual keyboard. Hand detection runs only on the first frame or when tracking is lost; subsequent frames use landmark prediction for the bounding box (Zhang et al., 2020).

### 3.2. Hand Landmark Detection

Hand-landmark detection is implemented in `src/hand_tracking.py` and invoked from the real-time loop in `src/main.py`. The detector is initialised once, before frame processing,

with the MediaPipe Tasks API: `num_hands = 1`, `min_hand_detection_confidence = 0.8`, and `min_tracking_confidence = 0.8`. This one-time initialisation avoids repeated model loading and keeps per-frame latency low.

For each frame, the system performs the following pipeline:

1. The webcam frame is captured in BGR format and mirrored horizontally (`cv.flip(frame, 1)`) so user motion appears natural on screen.
2. The frame is converted from BGR to RGB (`cv.cvtColor`), then wrapped as a MediaPipe `mp.Image` with `ImageFormat.SRGB`.
3. The hand landmarker runs inference (`detector.detect(mp_image)`) and returns up to one set of 21 landmarks (because `MAX_HANDS = 1`).
4. Landmark coordinates are denormalised from  $[0, 1]$  image space to pixel space:  $(x_i, y_i) = (\lfloor \hat{x}_i \cdot W \rfloor, \lfloor \hat{y}_i \cdot H \rfloor)$ .
5. To suppress jitter, all 21 landmarks are temporally smoothed with exponential blending:

$$\mathbf{p}_t^{(i)} = (1 - \alpha)\mathbf{p}_{t-1}^{(i)} + \alpha\hat{\mathbf{p}}_t^{(i)}, \quad \alpha = 0.35$$

where  $\hat{\mathbf{p}}_t^{(i)}$  is the current raw landmark position and  $\mathbf{p}_t^{(i)}$  is the filtered output.

6. From the smoothed landmarks, fingertip points are extracted (Thumb 4, Index 8, Middle 12, Ring 16, Pinky 20). The index fingertip is used for key hover selection and the thumb-index pair is used for pinch-click logic.
7. The hand skeleton is rendered by drawing predefined landmark connections and highlighting fingertips for debugging and visual feedback.

When no hand is detected, smoothing state and click-state variables are reset. This prevents stale landmark coordinates from triggering false interactions when the hand re-enters the frame. As in Figure 3, BlazePalm is invoked only when needed (initial detection or tracking loss), while most frames rely on landmark tracking, enabling stable real-time performance on CPU.

### 3.3. Virtual Keyboard Overlay

The keyboard layout is rendered as a semi-transparent overlay on the live camera feed. Each key is drawn as a rounded rectangle using `cv2.rectangle` and `cv2.putText`. The layout consists of five rows:

- Row 1: ` 1 2 3 4 5 6 7 8 9 0 - = Back
- Row 2: Tab Q W E R T Y U I O P [ ] \
- Row 3: Caps A S D F G H J K L ; ' Google
- Row 4: Shift Z X C V B N M , . / Shift
- Row 5: Space and Clear

When the index-finger tip hovers over a key, the key is highlighted in a brighter colour, providing visual feedback before the click is registered.

### 3.4. Click Detection Algorithm

A key press is registered when two conditions are met simultaneously:

1. The index-finger tip (landmark 8) is positioned within the bounding box of a key.
2. The Euclidean distance  $d$  between landmark 8 and landmark 4 falls below a threshold  $\theta$ :

$$d = \sqrt{(x_8 - x_4)^2 + (y_8 - y_4)^2} < \theta$$

The threshold  $\theta$  is empirically set to approximately 35 pixels for a  $1280 \times 720$  frame, corresponding to the natural closing motion of the index finger and thumb in a “pinch” gesture. The implementation supports three modes: `pinch`, `dwell`, and `pinch_or_dwell`. In the current configuration, `dwell` mode is enabled with a 650 ms dwell time and movement tolerance to prevent accidental repeated activations while hovering.

### 3.5. Key Injection

After a key is confirmed by the click detector, the event is passed to `handle_key_press()` in `src/input_handler.py`. A single `pynput.keyboard.Controller` instance is created at startup and reused for all emissions, so key events are sent directly to the active OS input focus.

The injection logic is stateful and updates three values on every press: typed text buffer, Caps Lock state, and Shift state. The behaviour is:

- **Regular characters:** output case is computed by `caps_on XOR shift_on`. The character is emitted with `controller.tap(out)` and appended to the on-screen text buffer.
- **Shift:** implemented as one-shot. Pressing Shift toggles `shift_on`; after one character is typed, Shift is automatically cleared.
- **Caps:** toggles persistent uppercase mode (`caps_on`).
- **Backspace / Tab / Space:** mapped to `Key.backspace`, `Key.tab`, and `Key.space`, respectively.
- **Clear:** clears only the internal text buffer.
- **Google:** opens a browser search using the current buffer as a URL-encoded query.

This design separates gesture recognition from host-input output: the vision subsystem decides “which key”, and the input handler decides “how to emit it”. As a result, the virtual keyboard works transparently in standard text fields without application-specific integration.

### 3.6. Results and Discussion

**Frame rate.** On a laptop equipped with an Intel Core i5 CPU (no dedicated GPU), the system processes frames at approximately 28–32 FPS at  $1280 \times 720$  resolution. MediaPipe’s optimisation of running the full palm detector only when needed is the primary reason real-time performance is achievable on CPU (Lugaresi et al., 2019).

**Detection accuracy.** Landmark detection is robust under standard indoor lighting. Accuracy degrades noticeably in low-light conditions or when the hand is partially occluded – limitations acknowledged by both Zhang et al. (2020) and Patle et al. (2025).

**Click precision.** The pinch-based click gesture was found to be intuitive for most users after



a short familiarisation period. In the default dwell configuration (650 ms), accidental repeated registrations are reduced because activation requires stable hover over a target key. False positives (unintended key presses) were rare during testing but can occur if the hand moves quickly across the keyboard layout.

**Typing speed.** Informal user trials suggest a typing speed of approximately 5–15 words per minute, substantially lower than physical keyboard benchmarks (50 wpm average; Kölsch & Turk, 2002). This gap is expected for a prototype touchless system, where the user must adapt motor habits learned for physical keyboards to an air-typing paradigm.

Figure 4 illustrates several example hand poses correctly detected by the pipeline across diverse lighting and background conditions, as reported in the MediaPipe Hands paper.

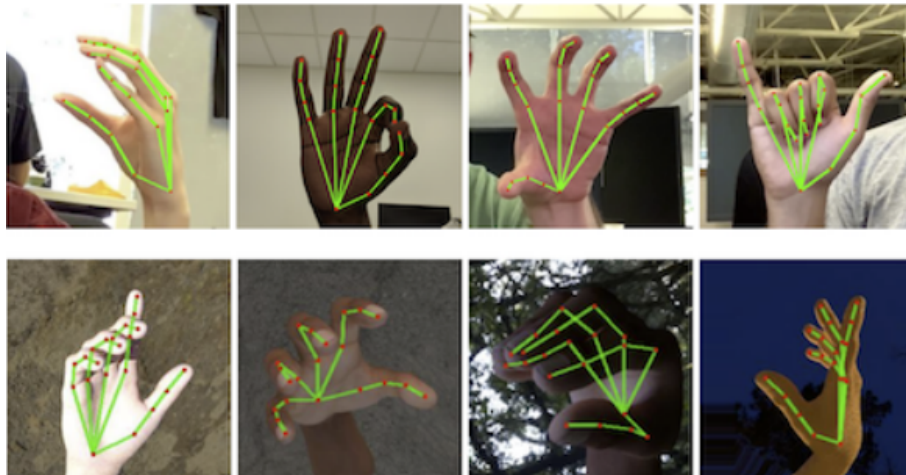


Figure 4: Examples of hand-landmark overlays produced by MediaPipe Hands across diverse conditions (Zhang et al., 2020).

## 4. Conclusion

This project demonstrates that a fully functional, camera-based virtual keyboard can be constructed from publicly available, open-source libraries running on commodity hardware. By combining MediaPipe Hands for real-time hand-landmark detection, OpenCV for overlay rendering, and `pynput` for keystroke injection, the system achieves interactive frame rates and operates as a transparent replacement for a physical keyboard in any text-input context.

The main **strengths** of the current implementation are its zero-hardware-cost input mechanism (requiring only a standard webcam), its cross-platform compatibility, and its modular architecture that makes it straightforward to swap gesture models or keyboard layouts.

The main **weaknesses** is sensitivity to lighting conditions and hand occlusion. The pinch-click gesture, while intuitive, requires users to learn a new motor habit and can be tiring during extended use.

**Future work** could address these limitations in several ways. First, replacing the distance-threshold click detector with a learned gesture classifier could improve reliability and enable richer interaction vocabularies. Second, integrating depth information from a depth-capable camera would allow more accurate three-dimensional finger-position estimation, reducing false

positives. Third, as Patle et al. (2025) suggest, combining the gesture-based keyboard with voice input and a context-aware modality-switching mechanism would create a more robust multimodal system suitable for accessibility applications – for example, allowing users with limited hand mobility to complement gesture input with voice commands. Finally, conducting formal user studies with larger participant groups and standardised typing-speed benchmarks would provide the quantitative evidence needed to assess real-world usability.

In summary, this project contributes a working proof-of-concept for contactless text input that aligns with the broader evolution of Human-Computer Interaction toward touchless, natural, and accessible interfaces (Patle et al., 2025; Kölsch & Turk, 2002).

## References

- Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 25, 120–125. <https://opencv.org>
- Kölsch, M., & Turk, M. (2002). *Keyboards without keyboards: A survey of virtual keyboards* (Technical Report UCSB-2002-21). University of California, Santa Barbara, Department of Computer Science.
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*. <https://arxiv.org/abs/1906.08172>
- Patle, B., Banothe, R., Kumbhare, B., & Laxane, Y. (2025). The future of touchless interfaces in human-computer interaction. *International Journal of Advanced Research in Science, Communication and Technology*, 5(9), 193–200. <https://doi.org/10.48175/IJAR SCT-24626>
- Rautaray, S. S., & Agrawal, A. (2015). Vision based hand gesture recognition for human computer interaction: A survey. *Artificial Intelligence Review*, 43(1), 1–54. <https://doi.org/10.1007/s10462-012-9356-9>
- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). MediaPipe Hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*. <https://arxiv.org/abs/2006.10214>